Prakash Thapa
D15A- 27

EXPERIMENT 03
Title: Application of Decision Tree and Random Forest for Classification

1. Dataset Source

Dataset Name: Bank Marketing Dataset

Source: Kaggle

Dataset Link:
https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset

Dataset Type: Real-world marketing dataset used to predict customer subscription to term deposit.

2. Dataset Description

Overview

The dataset contains marketing campaign data from a banking institution. The objective is to predict whether a customer will subscribe to a term deposit.

Dataset Size

Number of Records: ~11,162
Number of Features: 16 input features
Target Variable: deposit

Features Description

Feature	Description
age	Age of client
job	Type of job

marital	Marital status
education	Education level
default	Has credit in default
balance	Average yearly balance
housing	Has housing loan
loan	Has personal loan
contact	Communication type
day	Last contact day
month	Last contact month
duration	Last contact duration
campaign	Number of contacts during campaign
pdays	Days since last contact
previous	Number of contacts before campaign
poutcome	Outcome of previous campaign
deposit	Target variable (Yes/No)

Target Variable

deposit:

Yes → Customer subscribed
No → Customer did not subscribe

After encoding:

deposit_yes → 1 (Subscribed)
deposit_yes → 0 (Not Subscribed)

Dataset Characteristics

• Real-world marketing dataset
• Mixed numerical and categorical data
• Binary classification problem
• No significant missing values
• Moderate class imbalance

3. Mathematical Formulation of Algorithms

A. Decision Tree Classifier

Decision Tree splits dataset based on feature conditions.

It uses Gini Index or Entropy to measure impurity.

Gini Index:

$$\text{Gini} = 1 - \sum p_i^2$$

Where p_i = probability of class i

Goal: Minimize impurity at each split.

B. Random Forest Classifier

Random Forest is an ensemble method combining multiple Decision Trees.

Prediction:

Final Prediction = Majority Vote of all trees

Each tree is trained on:

• Random subset of data
• Random subset of features

This reduces variance and overfitting.

4. Algorithm Limitations

Decision Tree Limitations

• Prone to overfitting
• High variance
• Sensitive to small data changes

Not suitable for:
• Very small datasets
• Highly noisy datasets

Random Forest Limitations

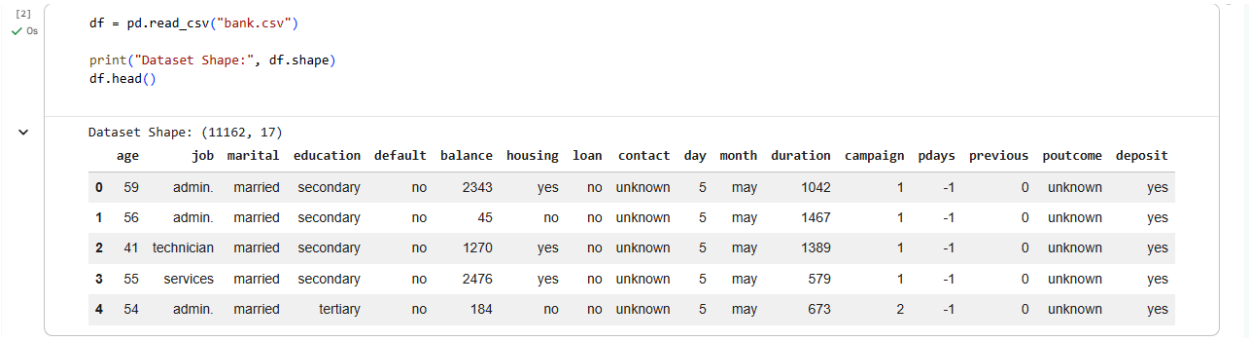
• Computationally expensive
• Less interpretable
• Requires more memory

Not ideal for:
• Real-time low-resource systems

5. Methodology / Workflow

Step 1: Dataset Loading

Loaded bank.csv using pandas.



```
[2]
✓ 0s
df = pd.read_csv("bank.csv")

print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (11162, 17)
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	deposit
0	59	admin.	married	secondary	no	2343	yes	no	unknown	5	may	1042	1	-1	0	unknown	yes
1	56	admin.	married	secondary	no	45	no	no	unknown	5	may	1467	1	-1	0	unknown	yes
2	41	technician	married	secondary	no	1270	yes	no	unknown	5	may	1389	1	-1	0	unknown	yes
3	55	services	married	secondary	no	2476	yes	no	unknown	5	may	579	1	-1	0	unknown	yes
4	54	admin.	married	tertiary	no	184	no	no	unknown	5	may	673	2	-1	0	unknown	yes

Step 2: Data Exploration

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11162 entries, 0 to 11161
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        11162 non-null  int64
 1   job        11162 non-null  object
 2   marital    11162 non-null  object
 3   education  11162 non-null  object
 4   default    11162 non-null  object
 5   balance    11162 non-null  int64
 6   housing    11162 non-null  object
 7   loan       11162 non-null  object
 8   contact    11162 non-null  object
 9   day        11162 non-null  int64
 10  month      11162 non-null  object
 11  duration   11162 non-null  int64
 12  campaign   11162 non-null  int64
 13  pdays      11162 non-null  int64
 14  previous   11162 non-null  int64
 15  poutcome   11162 non-null  object
 16  deposit    11162 non-null  object
dtypes: int64(7), object(10)
```

```
df.isnull().sum()
```

	0
age	0
job	0
marital	0
education	0
default	0
balance	0
housing	0
loan	0
contact	0
day	0
month	0
duration	0
campaign	0
pdays	0

```
df.describe()
```

	age	balance	day	duration	campaign	pdays	previous
count	11162.000000	11162.000000	11162.000000	11162.000000	11162.000000	11162.000000	11162.000000
mean	41.231948	1528.538524	15.658036	371.993818	2.508421	51.330407	0.832557
std	11.913369	3225.413326	8.420740	347.128386	2.722077	108.758282	2.292007
min	18.000000	-6847.000000	1.000000	2.000000	1.000000	-1.000000	0.000000
25%	32.000000	122.000000	8.000000	138.000000	1.000000	-1.000000	0.000000
50%	39.000000	550.000000	15.000000	255.000000	2.000000	-1.000000	0.000000
75%	49.000000	1708.000000	22.000000	496.000000	3.000000	20.750000	1.000000
max	95.000000	81204.000000	31.000000	3881.000000	63.000000	854.000000	58.000000

Checked:

- Dataset shape
- Data types
- Missing values
- Statistical summary

Step 3: Data Preprocessing

• Converted categorical features using One-Hot Encoding
• Encoded target variable deposit
• Created numerical feature matrix

Step 4: Feature and Target Separation

X = All input features
y = deposit_yes

```
[9]
✓ 0s
X = df_encoded.drop("deposit_yes", axis=1)
y = df_encoded["deposit_yes"]

print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
Feature Shape: (11162, 42)
Target Shape: (11162,)
```

Step 5: Train-Test Split

70% → Training data
30% → Testing data

Using train_test_split()

```
[10]
✓ 0s
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("Training Features:", X_train.shape)
print("Testing Features:", X_test.shape)
```

```
Training Features: (7813, 42)
Testing Features: (3349, 42)
```

Step 6: Decision Tree Training

Used:

DecisionTreeClassifier()

Trained model and evaluated using:

• Accuracy
• Confusion Matrix
• Classification Report

```
[10]
✓ 0s
```

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("Training Features:", X_train.shape)
print("Testing Features:", X_test.shape)
```

```
Training Features: (7813, 42)
Testing Features: (3349, 42)
```

```
[12]
✓ 0s
```

```python
print("Confusion Matrix:\n")
print(confusion_matrix(y_test, y_pred_dt))

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_dt))
```

```
Confusion Matrix:

[[1398  344]
 [ 376 1231]]

Classification Report:

              precision    recall  f1-score   support

       False       0.79      0.80      0.80      1742
        True       0.78      0.77      0.77      1607

    accuracy                           0.79      3349
   macro avg       0.78      0.78      0.78      3349
weighted avg       0.78      0.79      0.78      3349
```

Step 7: Random Forest Training

Used:

RandomForestClassifier(n_estimators=100)

Evaluated using:

• Accuracy
• Confusion Matrix
• Precision

• Recall
• F1-score

```
[13]
✓ 1s
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.8471185428486115
```

```
[14]
✓ 0s
print("Confusion Matrix:\n")
print(confusion_matrix(y_test, y_pred_rf))

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_rf))
```

```
Confusion Matrix:

[[1440  302]
 [ 210 1397]]

Classification Report:

              precision    recall  f1-score   support

       False       0.87      0.83      0.85      1742
        True       0.82      0.87      0.85      1607

    accuracy                           0.85      3349
   macro avg       0.85      0.85      0.85      3349
weighted avg       0.85      0.85      0.85      3349
```

Step 8: Feature Importance Analysis

Extracted feature importance from Random Forest.

Top features typically:

• duration
• balance
• age
• campaign
• pdays

```
[15]
✓ 0s
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Decision Tree Accuracy: 0.78501045088086
Random Forest Accuracy: 0.8471185428486115
```

Workflow Diagram

Dataset → Preprocessing → Encoding → Train-Test Split → Train Decision Tree → Train Random Forest → Compare Performance → Feature Importance Analysis

```
[16]
✓ 0s
import pandas as pd
import matplotlib.pyplot as plt

# Get feature importance
importances = rf_model.feature_importances_

# Create DataFrame
feature_importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importances
})

# Sort values
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

# Show top 10 important features
print(feature_importance_df.head(10))
```

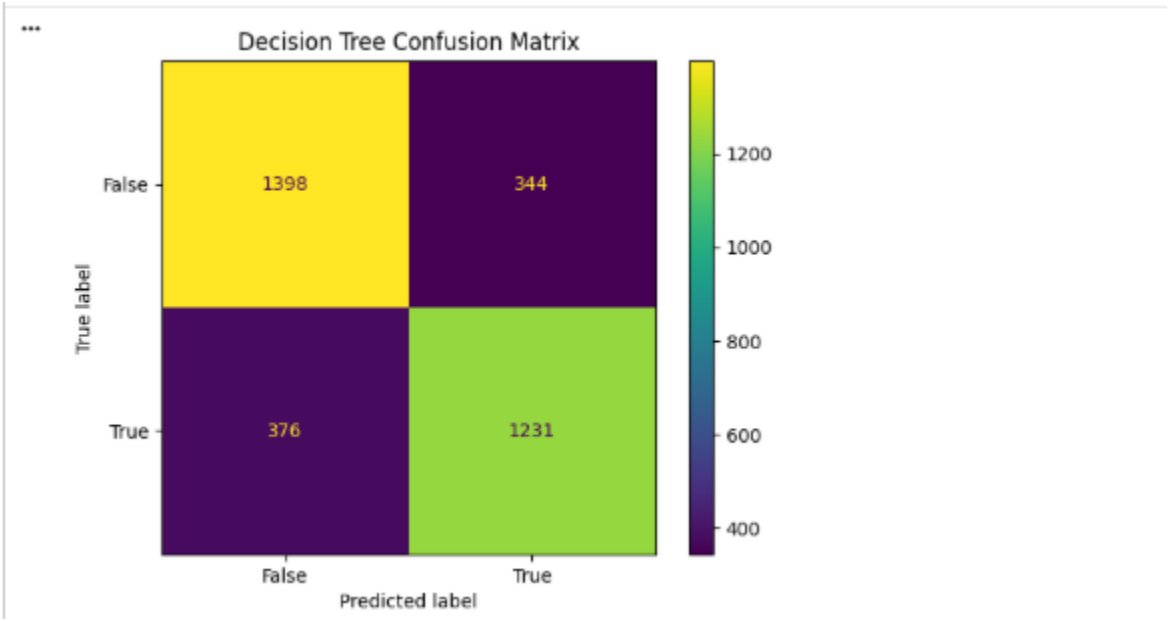
```
                Feature  Importance
3              duration    0.339204
1               balance    0.083095
0                   age    0.078536
2                   day    0.071177
5                 pdays    0.036839
4              campaign    0.036785
27   contact_unknown    0.033536
40  poutcome_success    0.031314
24       housing_yes    0.025720
6             previous    0.021563
```

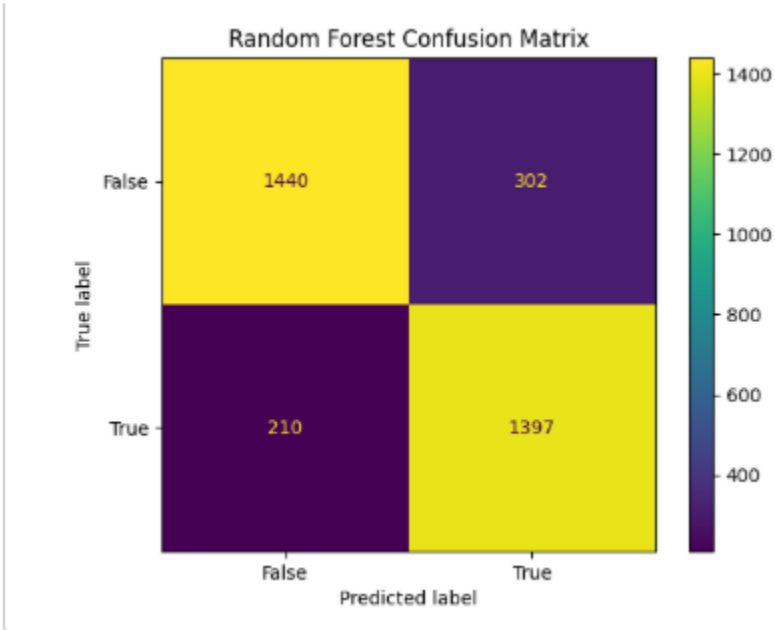
Visualization

Decision Tree and Random Forest Classification (Bank Marketing Dataset)

1. Confusion Matrix Analysis

Confusion matrices were generated for both the Decision Tree and Random Forest classifiers to evaluate classification performance.





Observations:

- The majority of predictions were concentrated along the diagonal, indicating correct classifications.

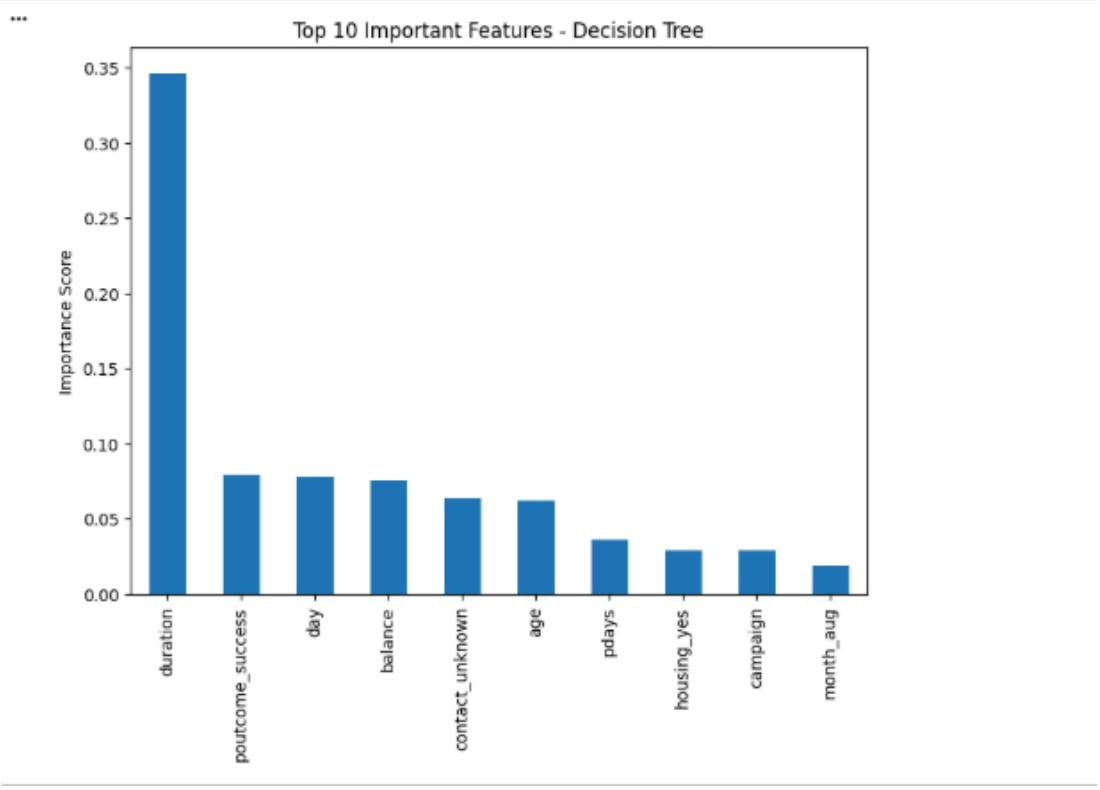
- The Random Forest model exhibited fewer false positives and false negatives compared to the Decision Tree model.

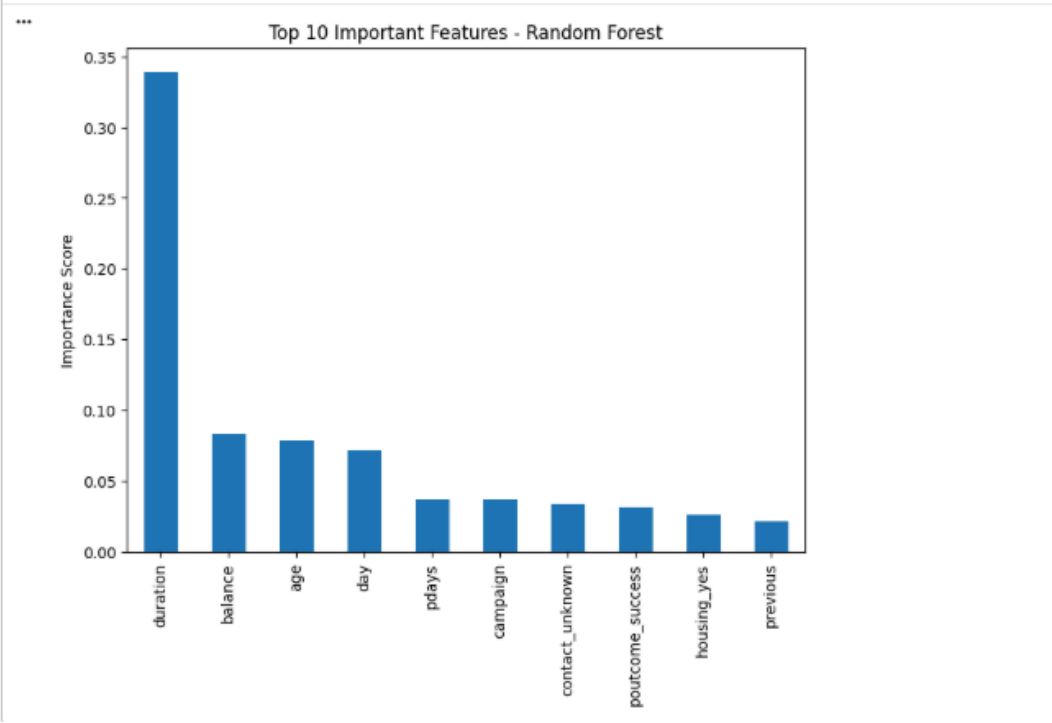
Interpretation:

- A higher number of true positives and true negatives indicates better classification performance.
- Random Forest demonstrated improved generalization ability.
- The Decision Tree model showed relatively higher misclassification, suggesting potential overfitting.

2. Feature Importance Analysis

Feature importance plots were generated for both models to identify the most influential predictors in the classification task.





Key Influential Features Identified:

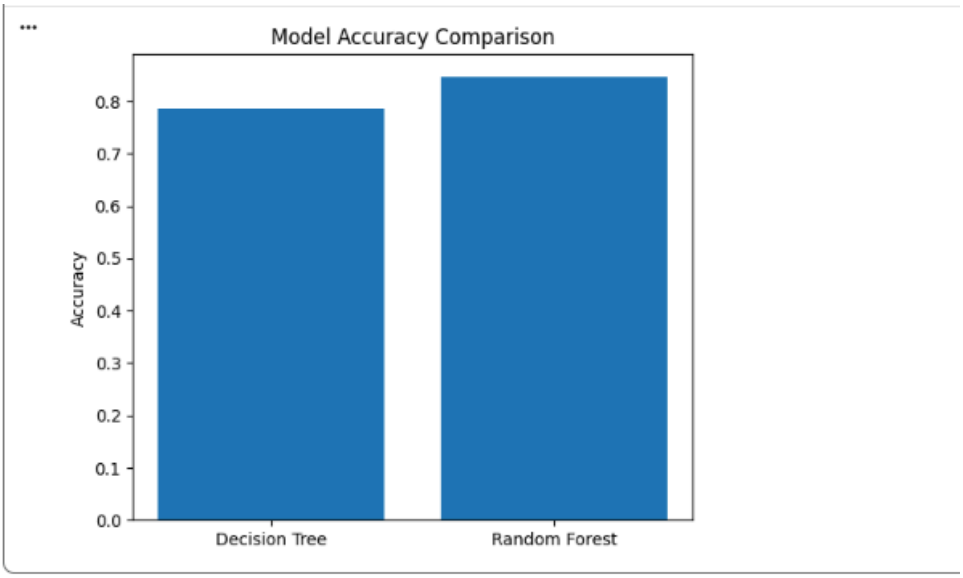
- `duration`
- `poutcome_success`
- `balance`
- `age`

Interpretation:

- The duration of the call was the most significant predictor of term deposit subscription.
- Random Forest provided a more stable and balanced distribution of feature importance.
- Decision Tree showed higher variance in importance distribution, often emphasizing a small number of dominant features.

3. Model Accuracy Comparison

A comparative bar chart was plotted to evaluate the overall accuracy of both models.



Observations:

- Random Forest achieved higher accuracy compared to Decision Tree.
- The performance gap indicates the advantage of ensemble learning techniques.

Interpretation:

- Random Forest improves predictive performance by combining multiple decision trees and reducing variance.
- Decision Tree, while simpler and interpretable, is more prone to overfitting.

6. Performance Analysis

Evaluation Metrics Used

Accuracy:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score:

$$F_1 = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

Observations

- Random Forest performed better than Decision Tree
- Decision Tree showed slight overfitting
- Random Forest gave more stable and higher accuracy
- Feature duration had highest importance

Interpretation

Longer call duration increases probability of subscription.

Random Forest reduces overfitting by combining multiple trees.

7. Hyperparameter Tuning

Decision Tree Parameters

max_depth
min_samples_split
criterion (gini / entropy)

Example:

DecisionTreeClassifier(max_depth=5)

Effect:

• Controls overfitting
• Improves generalization

Random Forest Parameters

n_estimators → Number of trees
max_depth → Tree depth
max_features → Number of features per split

Example:

RandomForestClassifier(n_estimators=200, max_depth=10)

Impact:

• Higher n_estimators → More stability
• Proper max_depth → Prevents overfitting

GridSearchCV can be used for optimal tuning.

Final Conclusion

This experiment successfully implemented:

Decision Tree Classifier
Random Forest Classifier
Feature Importance Analysis

Results showed:

• Random Forest outperformed Decision Tree
• Ensemble learning improves classification accuracy
• Feature importance provides insight into customer behavior

Learning Outcome:

Understanding of tree-based models, ensemble methods, and their effectiveness in real-world classification problems.